JobMatch: An Agentic Multi-Role Platform for Explainable Job–Candidate Matching

Harsh Kashyap[1*] and Taranumpreet Kaur Wasu[1]

[1*]Thapar Institute of Engineering and Technology, Patiala, India.

Abstract

Job seekers and recruiters frequently receive ranked lists without clear attribution of profile ownership, scoring rationale, or control over consequential actions. We present JobMatch, a multi-agent recruitment platform in which Candidate and Employer agents maintain role-specific profiles and embeddings, while a read-only Matchmaking Agent scores and explains candidate–job pairs. Agents coordinate through an event bus; when a resume or job description changes, the owning agent updates vector representations and notifies the Matchmaking layer to refresh stale rankings. Separate candidate, employer, and admin portals keep save, apply, and shortlist decisions with human users. For deployment, the Employer Agent can optionally refresh job postings from an external provider API; all offline metrics in this paper use the frozen demo corpus.

The Matchmaking agent combines semantic similarity, skill overlap, and experience, compensation, and remote compatibility into a fixed composite score with structured explanations—assistive ranking, not autonomous hiring. We evaluate offline on 30 resumes, 15 job postings, and 47 graded relevance pairs using macro-averaged P@5, R@5, and nDCG@5 at $\boldsymbol{K=5}$. Compared methods include TF–IDF and BM25 baselines, dense retrieval, reciprocal rank fusion, learned fusion, and cross-encoder reranking. Multimodal soft-embed and learned fusion reach nDCG@5 of 0.969 and 0.968 in committed artifacts; composite ablation reaches 0.949 under the portal-default weight vector. Cross-encoder reranking reduces nDCG@5 by 0.108 at $\boldsymbol{\sim 141.7}$ ms/query and remains disabled in the default portal. Hard-negative mining yields 150 pairs with zero label conflicts; a synthetic fairness audit flags 7/10 fabricated pairs—an engineering check, not demographic validation.

Benchmark drivers and regression tests reproduce reported metrics. We document multi-agent architecture, implementation, quality metrics, and results as an integrated research prototype, not a validated production hiring system.

Keywords: multi-agent systems, recruitment platforms, explainable matching, job–candidate ranking, hybrid retrieval

1 Introduction

Online hiring markets increasingly depend on ranking systems that match candidates to jobs at scale, yet the reasons behind those rankings are often opaque. Candidates see ordered lists without a clear link to resume fields or job requirements, while employers see shortlists change after role edits without a visible account of which requirement moved the ranking. This opacity makes it difficult for either side to answer a practical question: *why was this person shown this role, and what would need to change for the ranking to differ?*

We address this problem through **JobMatch**, a research prototype for explainable job–candidate matching. The system separates the market into three logical agents. A **Candidate Agent** owns resumes, profile updates, and candidate snapshots. An **Employer Agent** owns job descriptions, requirement updates, and job snapshots. A neutral **Matchmaking Agent** reads those snapshots, scores candidate–job pairs, and returns ranked recommendations with structured explanations. When either side edits its data, the owning agent refreshes its snapshot and notifies the matching layer so stale rankings can be invalidated without cross-agent writes.

This architecture follows human–agent systems research that positions agents as decision support with calibrated trust, not autonomous hiring authority [1]. JobMatch does not automate hiring decisions. It parses resumes and job descriptions into structured states, exposes component-level reasons for each match, and leaves save, apply, shortlist, and reject decisions to users. The prototype includes separate candidate, employer, and administrator portals, but the core research question is narrower: whether role-separated agents can support auditable, reproducible, and explainable ranking.

The system is evaluated offline on a fixed demo corpus of 30 resumes, 15 jobs, and 47 graded relevance pairs. We compare lexical baselines, dense embedding methods, hybrid semantic–skill rankers, fusion variants, ablations, latency, and fairness/explainability sanity checks. Hybrid rankers reach nDCG@5 up to 0.969 on this corpus; the portal defaults to a fixed composite score whose six components map directly to user-visible explanations. The results should be read as evidence for an assistive ranking prototype, not as evidence of production hiring impact.

This paper makes the following contributions:

- A **Candidate/Client Agent** that ingests resumes, maintains versioned candidate state, and updates candidate embeddings on confirmed saves.
- An **Employer Agent** that ingests job descriptions, maintains versioned job state, and optionally refreshes postings from an external provider API.
- A **Matchmaking Agent** that scores candidate–job pairs read-only and returns ranked lists with structured explanations.
- An **event-driven communication model** in which profile updates invalidate stale rankings without cross-agent writes to foreign stores.
- A **role-separated portal layer** demonstrating end-to-end candidate, employer, and administrator workflows.
- An **offline evaluation** on a fixed labeled corpus with benchmark comparisons, ablations, and a continuous-integration regression gate.

The remainder of the paper is organized as follows. Section 2 reviews recruitment systems, AI agents in hiring, and related matching approaches. Section 3 presents the multi-agent architecture (Candidate, Employer, and Matchmaking agents; communication; end-to-end workflow). Section 4 summarizes implementation details. Section 5 defines quality metrics, similarity scores, and evaluation criteria. Section 6 reports benchmark results and discussion. Section 7 concludes and outlines future scope.

2 Literature Review

Recruitment technology, agentic software, and information retrieval have each advanced independently, yet deployed hiring tools rarely combine all three in a traceable architecture. We organize prior work into four literature themes—recruitment systems, multi-agent coordination, semantic representation, ranking and fusion—then state the gaps those literatures leave open and how JobMatch is positioned relative to them. Numeric results from our evaluation appear in Section 6; metric definitions in Section 5.

2.1 Recruitment and job-matching systems

Enterprise hiring still depends heavily on manual screening: recruiters read resumes, compare them to written requirements, and decide who advances. At scale this workflow is slow, inconsistent across reviewers, and weakly auditable when a candidate asks why they were excluded. Most production stacks therefore automate the first cut through keyword filters, structured application forms, and applicant tracking systems (ATS).

Keyword-based filtering treats a resume as a bag of terms matched against job requirements. Lexical retrieval methods such as TF–IDF weighting and BM25 remain the workhorse for this pattern because they are fast, interpretable at the token level, and easy to deploy over large applicant pools [2, 3]. Their limitation is semantic brittleness: synonyms, paraphrases, and transferable skills that are not spelled out literally can be missed, while keyword stuffing can inflate apparent fit. Structured skill fields and Boolean search rules reduce noise but still encode fit as a static rule set rather than a graded comparison between two evolving profiles.

ATS platforms extend ingestion with workflow state—application status, interview stages, compliance logging—but they often treat matching as a side feature bolted onto document storage. Candidates frequently experience opaque ranking: a profile passes a filter without knowing which requirement mattered, or disappears after a re-rank with no explanation. Recruiters face the symmetric problem when shortlists shift without a visible link between job-text edits and ranking changes. These workflow limitations motivate systems that separate *profile ownership* from *scoring*, and that return reasons a human can inspect before any hire/no-hire action is taken. Early work on job recommendation already recognized bilateral matching, scoring candidate–job pairs from both sides' utility [4]. Reciprocal recommendation research further shows that when acceptance is costly—as in job applications—explanations must address both parties, not only the recommendation receiver [5]. Fairness-aware reciprocal models balance multi-objective fit and market equilibrium between applicants and employers [6].

2.2 AI agents and multi-agent systems

Agent-based software assigns bounded roles—perception, memory, planning, action—to modules that coordinate through messages or shared events rather than through one monolithic controller [7]. Three agent patterns are relevant to hiring.

Personal agents. Personal or client-side agents maintain a durable model of a user's goals and history, updating that model as new documents arrive [7]. In hiring, the natural analogue is a representative that ingests resumes, tracks skill and experience revisions, and speaks for the candidate without exposing raw edits to the employer's datastore.

Task-specific agents. Task-specific agents encapsulate one domain workflow—parsing, validation, or recommendation—with explicit inputs and outputs. Employer-side job-ingestion agents fit this pattern: they normalize postings, extract requirements, and version job records when descriptions change. Some agentic hiring proposals emphasize autonomous interview bots or negotiation agents. JobMatch instead uses task-specific agents for *profile maintenance*, not for autonomous selection.

Collaborative agents and human-in-the-loop support. Collaborative multi-agent systems decompose a problem so that each agent owns part of the state and a coordinator integrates read-only views for decision support [7]. Human-in-the-loop designs keep consequential actions—apply, reject, shortlist—with people while software proposes ranked options and explanations [8]. This differs from fully automated ranking pipelines that update applicant status without an explicit user action on each match. Following the Artifacts & Agents view, shared infrastructure can mediate coordination without cross-agent state mutation [9].

JobMatch adopts the collaborative pattern: Candidate and Employer agents own their respective profiles and vector stores; a read-only Matchmaking agent scores cross-side pairs and emits structured explanations (Section 3). An in-process event bus synchronizes profile updates and match-cache invalidation without cross-agent mutation of stored records (Section 4), treating posting repositories and embedding indexes as environment artifacts rather than foreign agent stores.

2.3 Semantic search and representation learning

Job–candidate matching is, at core, a cross-document retrieval problem: given a resume (query), rank jobs (documents), or the reverse for employer-initiated search. Semantic search replaces exact term overlap with learned representations that map text into a shared vector space [2].

Embeddings. Dense encoders map resumes and job descriptions to fixed-dimensional vectors so that related texts lie nearby even when they share few tokens. Sentence-level models such as Sentence-BERT train siamese encoders for semantic similarity and have become a default building block for short-text matching [10]. In JobMatch, snapshot embeddings are computed from canonical resume and job-description text (Section 4.3).

Dense retrieval and semantic similarity. Dense retrieval ranks candidates or jobs by vector similarity—typically cosine—over the full corpus or over an approximate

nearest-neighbor shortlist [2, 10]. Compared with lexical baselines, dense retrieval captures paraphrase and topical relatedness but can underweight explicit skill tokens that recruiters treat as hard requirements. Hybrid systems therefore retain both signals rather than replacing keywords entirely [2, 10].

Many job-matching prototypes evaluate a single embedding model or similarity function on a static labeled set. Fewer prototypes expose the same representation path to live profile edits on both market sides while reporting offline retrieval metrics on identical scoring code.

2.4 Ranking and recommendation

Once candidates and jobs are represented, the system must *rank* pairs and *evaluate* ranking quality—topics well developed in information retrieval and recommender-systems research.

Ranking metrics. Standard offline metrics include precision and recall at cutoff K, mean reciprocal rank (MRR), normalized discounted cumulative gain (nDCG@K), and mean average precision (MAP) [2]. Precision@K measures shortlist purity; recall@K measures coverage of labeled relevant items; nDCG@K rewards placing higher-grade relevance labels earlier; MRR emphasizes the rank of the first relevant result. Graded nDCG is appropriate when relevance is partial (e.g., a job is a plausible but not ideal fit). We adopt these metrics at K=5 on a labeled demo corpus (Section 5).

Hybrid retrieval. Hybrid rankers combine lexical scores, dense semantic similarity, and structured features such as skill overlap or experience gaps [2, 3, 10]. A common pattern is a weighted sum of semantic and skill channels (multimodal blending) or a post-hoc constraint filter on top of a semantic shortlist. JobMatch extends this with a multi-component composite score—semantic, skills, title overlap, experience, compensation, and remote preference—so that each channel remains visible in portal explanations (Section 5). Reciprocal job recommenders increasingly balance such multi-objective fit and fairness across market sides [6], while reciprocal explanation methods argue that both parties should see why a match is proposed when acceptance is costly [5].

Fusion methods. When multiple rankers produce different orderings, fusion methods merge lists without retraining the underlying encoders. Reciprocal rank fusion (RRF) combines ranked outputs using reciprocal rank weights and remains stable when individual retrievers make complementary errors [11]. Learned fusion and hierarchical blending fit scalar weights or classifiers over component scores. JobMatch implements fixed-weight composite scoring for portal defaults, plus optional RRF, learned logistic fusion, Platt calibration [12], and cross-encoder reranking for offline comparison (Section 4, Section 6).

2.5 Research gaps

Across the recruitment, agent, and retrieval literatures, several gaps recur in job-matching demos and prototypes.

Algorithm-only focus. Many systems optimize a matching formula—embedding similarity, skill Jaccard, or a learned re-ranker—while treating ingestion, profile versioning, and user-facing workflow as out of scope. Evaluation then proceeds on frozen corpora disconnected from the code path users would actually trigger when they edit a resume or repost a job.

Single-sided or monolithic control. Candidate-side recommender demos are more common than symmetric platforms where *both* sides maintain agents with separate ownership. Monolithic services that store and score all records in one module obscure accountability: users cannot tell which component changed a rank or which side's edit caused invalidation.

Limited explainability and evaluation depth. Ranking scores alone do not satisfy recruiter or candidate trust; explainable ML methods attribute predictions to input features [13]. JAAMAS work argues for multi-level, stakeholder-tailored agent explanations [14] and catalogs explainability goals in human–agent decision support [1]. In hiring, explanations should reference skills, experience gaps, and constraint channels that align with the score breakdown, and reciprocal settings may require rationales visible to both sides [5]. Yet many job-matching papers report nDCG or accuracy without explanation-quality audits, calibration checks [12], or fairness sanity tests on controlled profile variants [13]. Demo corpora are often small and manually labeled, which is appropriate for reproducible research but rarely accompanied by regression gates tied to production scoring code.

2.6 Positioning of JobMatch

JobMatch addresses these gaps as an integrated research prototype rather than as a standalone retrieval formula.

Multi-agent framing. The system decomposes hiring into three bounded agents—Candidate, Employer, and Matchmaking—coordinated by typed events (Section 3–Section 4). Profile mutation stays with the owning agent; matching reads versioned snapshots only. This design makes ownership, cache invalidation, and audit trails first-class, not emergent properties of a single service.

Candidate-side and employer-side portals. Separate React portals enforce role separation: candidates manage resumes, saved jobs, and applications; employers manage postings, reverse candidate search, and applicant lists; administrators inspect agent status, run evaluation hooks, and review fairness summaries (Section 4.2). Consequential actions remain explicit user clicks—save, apply, dismiss—consistent with human-in-the-loop decision support.

Matchmaking agent and explainable composite ranking. The Matchmaking agent combines semantic, skill, and constraint channels into a fixed composite score with structured `MatchExplanation` payloads (Section 5). Rule-based explainers are evaluated offline for faithfulness and specificity (Section 5.5); optional calibration and feedback boosts exist but are disabled in default portal settings.

Empirical evaluation on shared code paths. Offline benchmarks call the same scoring core as the live gateway: lexical and dense baselines, multimodal and composite strategies, RRF and learned fusion, ablations, hard-negative sanity checks, synthetic fairness counterfactuals, and a continuous-integration regression gate on nDCG@5

(Section 5–Section 6). We do not claim state-of-the-art hiring accuracy on a large production corpus; the contribution is traceable multi-agent architecture, assistive explainable ranking, and reproducible measurement on a controlled labeled set tied to the deployed prototype.

From gaps to design. Four recurring gaps map directly onto JobMatch components:

- *No candidate-side profile ownership* → the Candidate/Client Agent, which owns resume-derived state and embeddings.
- *No employer-side ownership* → the Employer Agent, which owns job postings and requirement snapshots.
- *No explainable, multi-signal matching* → the read-only Matchmaking Agent and its structured composite explanations.
- *No real-time invalidation when profiles change* → the in-process event bus that refreshes stale rankings without cross-agent writes.

To address these four gaps, JobMatch introduces the multi-agent architecture presented in Section 3.

3 Architecture of the Multi-Agent System

JobMatch organizes hiring support around three collaborating agents and a shared representation layer. Figure 1 shows the high-level layout; Figures 2 and 3 expand the candidate and employer agents; Figure 4 traces the employer workflow end to end; Figure 5 details the Matchmaking agent internals; Figure 6 traces the candidate lifecycle end to end. Figure 7 (at the end of this section) revisits the high-level architecture with each lane expanded into its portal and agent responsibilities, and adds the Admin/Evaluation Console. Three design principles guide the layout: **separation of ownership** (each market side controls its own records), **read-only matchmaking** (scoring never edits profiles or acts on behalf of users), and **human-in-the-loop decisions** (save, apply, contact, and shortlist remain explicit user actions). Concrete libraries, APIs, and parameters appear in Section 4; score formulas and evaluation metrics in Section 5.

3.1 Candidate/Client Agent

The **candidate-side agent** (also referred to as the client-side agent in our implementation) represents the job seeker. Its responsibility is to turn resumes and profile edits into a maintained *candidate state* that the rest of the system can trust.

When a user uploads or revises a resume, this agent assists with parsing, normalizes skills into a shared vocabulary, and registers a versioned snapshot of the person's skills, experience, compensation expectations, and remote preferences. It also maintains the editable portal profile the user sees in forms. The agent publishes a profile-updated signal whenever a new snapshot is ready so that stale match lists can be refreshed.

The candidate agent does *not* own job postings, write into employer records, or decide which applications are sent.

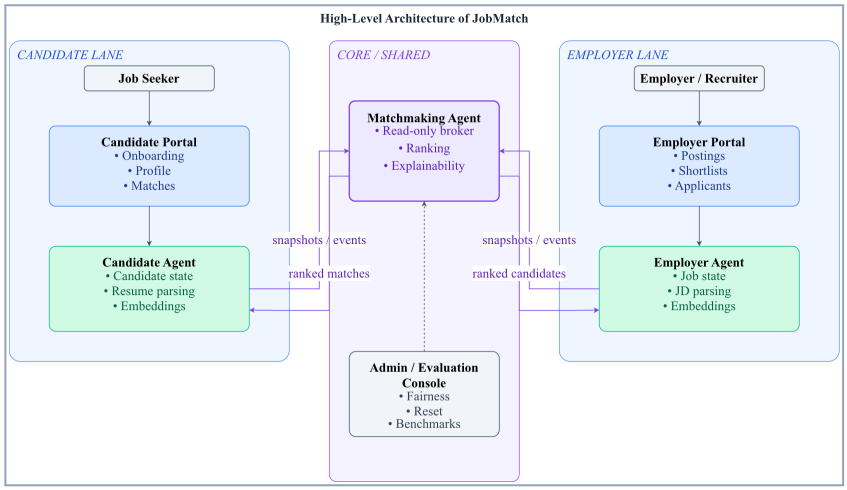

Fig. 1: High-level view of JobMatch: two market-side agents own candidate and job state; a read-only Matchmaking agent scores pairings and returns explained rankings to role-specific portals. Snapshots and invalidation events connect owning agents to the matcher; an admin console supports evaluation without crossing ingestion boundaries.

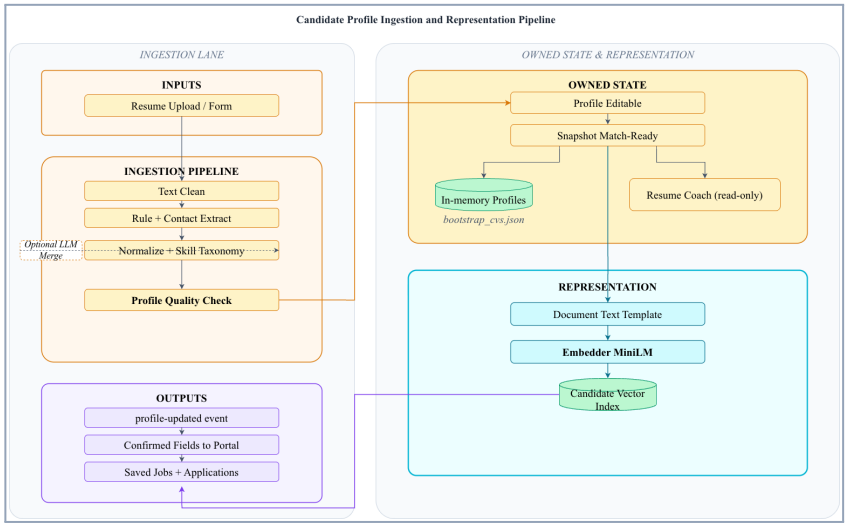

Fig. 2: Internal structure of the Candidate/Client agent: document ingestion (clean, rule extract, optional LLM merge), skill normalization, profile quality check, editable profile versus match-ready snapshot, embedding, and side outputs (events, activity store, resume coach).

3.2 Employer Agent

The **employer-side agent** represents the hiring organization. It accepts job descriptions and requirement updates, structures them into a maintained *job state*, and keeps that state aligned with what recruiters edit in the employer portal.

For each posting, the agent registers a snapshot with required and preferred skills, experience bounds, compensation band, remote policy, and descriptive text suitable for comparison against candidates. Like its counterpart on the candidate side, it emits a job-updated signal when a snapshot changes so match rankings can be invalidated without cross-agent writes.

The employer agent does *not* manage candidate profiles or override match scores. Shortlisting and applicant review remain human decisions supported by ranked suggestions.

Optional external job feed. When enabled at deployment time, the Employer agent can also ingest paginated job listings from an external provider API. Each raw record is normalized to the same canonical job schema used for portal postings, deduplicated by stable identifier, and written to a local snapshot file for restart recovery. A sync operation replaces the in-memory employer corpus and re-embeds job snapshots; the frozen `jobs.json` demo seed remains the default for offline evaluation reported in Section 6. The Matchmaking agent consumes the refreshed snapshots read-only; it does not call the external provider directly.

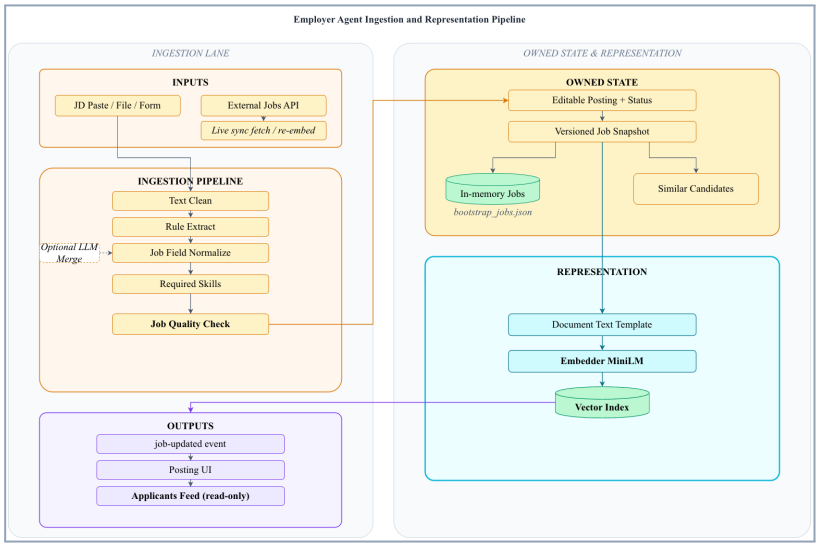

Fig. 3: Internal structure of the Employer agent: JD ingestion (clean, rules, optional LLM), job quality check, posting status lifecycle, optional external feed sync, job snapshots, and side outputs (events, applicants feed, similar candidates).

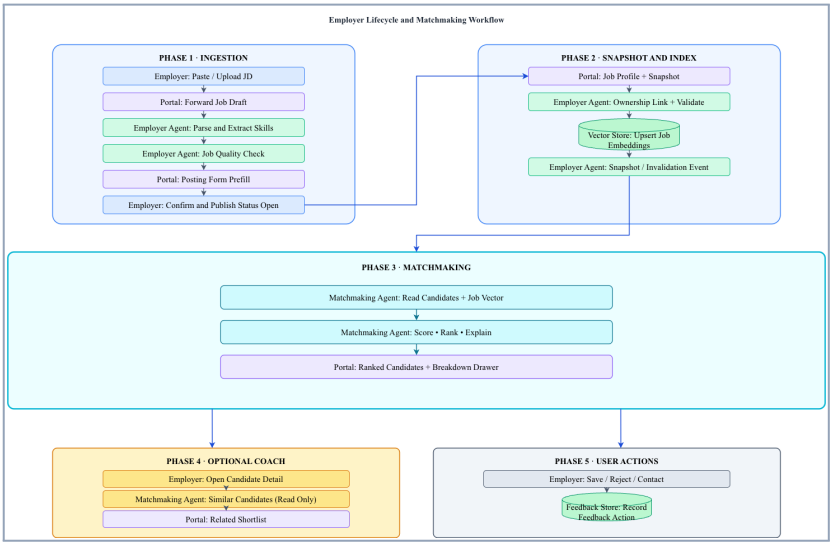

Fig. 4: Employer workflow: JD ingestion, job quality check, posting confirmation with status open, snapshot registration, reverse candidate matching with explanation drawer, feedback actions (save/reject/contact), and applicants page review.

3.3 Matchmaking Agent

The **Matchmaking Agent** is a neutral broker between the two market sides. Given a candidate snapshot and a set of job snapshots—or the reverse direction for employer-initiated search—it scores each feasible pairing, attaches structured explanations, and returns a ranked list.

It reads snapshots by identifier only; it never mutates candidate or job stores. It may fuse multiple retrieval signals and optional reranking stages when configured. The default portal path favors a fixed composite strategy whose components users can inspect (Section 4, Section 5). When profile or job events indicate that inputs changed, the matchmaking layer drops cached rankings for the affected entity and recomputes on the next request.

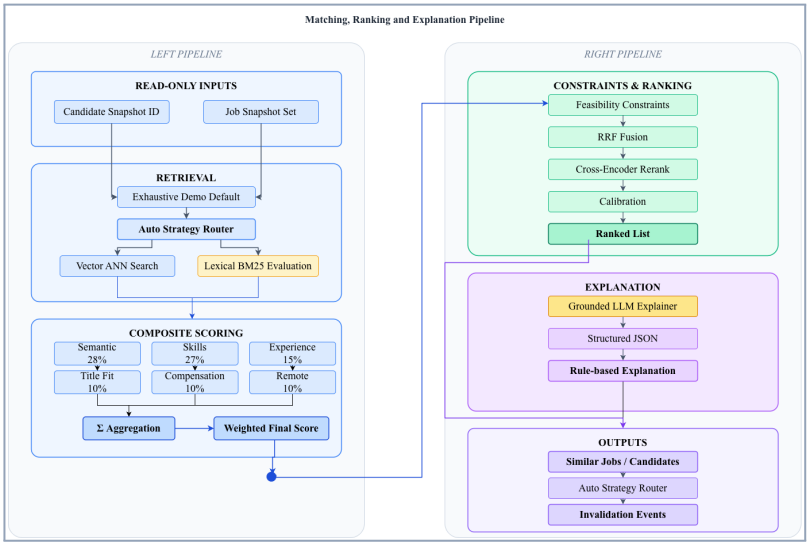

Fig. 5: Internal structure of the read-only Matchmaking agent: retrieval (exhaustive or ANN; lexical eval-only), composite scoring with six weighted components including title fit, feasibility constraints, optional fusion/rerank/calibration, rule or LLM explanations, and session invalidation events.

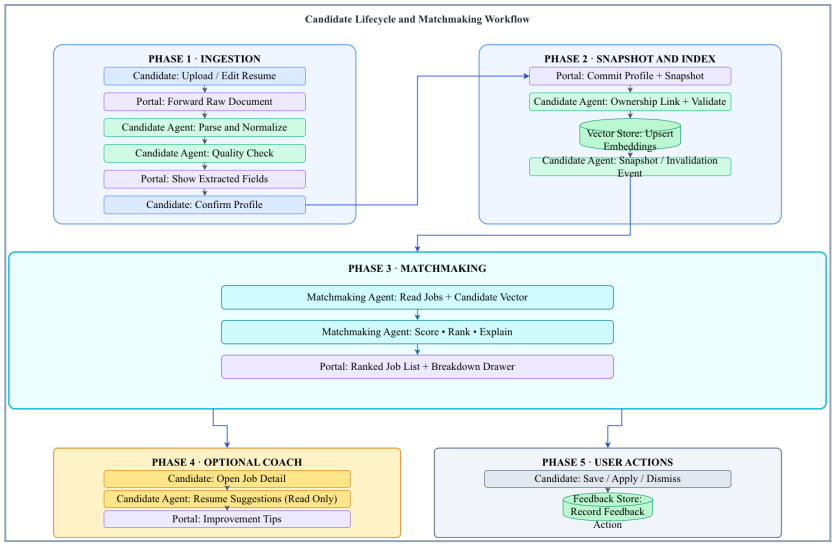

Fig. 6: Candidate lifecycle and matchmaking workflow in five phases: (1) ingestion with assisted parsing and quality check; (2) snapshot registration with ownership link, vector upsert, and invalidation event; (3) read-only matchmaking that scores, ranks, and attaches explanations; (4) optional read-only resume coach; (5) explicit save/apply/dismiss feedback recorded to the feedback store.

3.4 Agent Communication and State Sharing

JobMatch uses exactly three cooperating agents—Candidate/Client, Employer, and Matchmaking (Figures 1 and 5). Sections 3.1–3.3 define their roles; here we specify what each keeps private, what crosses the boundary, and how updates propagate. The three design principles of Section 3—separation of ownership, read-only matchmaking, and human-in-the-loop decisions—are stated once and not repeated per agent.

3.4.1 Ownership and state

Each owning agent maintains two layers: a **profile**, the editable portal record the user can revise at any time, and a **snapshot**, a versioned match-ready view frozen when the user confirms a save (normalized skills, constraint values, document identity, and a comparable text representation with its embedding). Only snapshots enter ranking, so people can edit without immediately rewriting history. The Candidate Agent owns resume-derived profiles, preferences, normalized skills, and candidate state; the Employer Agent owns the parallel job objects. The Matchmaking Agent owns neither store: it owns the retrieval-and-ranking workflow and the transient, session-scoped output (cached lists, last-computed scores, and explanation payloads), and keeps no third copy of full resumes or job descriptions.

3.4.2 Shared state

Agents share only what fair comparison and timely refresh require. On a match request, **snapshots and identifiers** cross to the Matchmaking Agent (the query-side snapshot plus the candidate or job snapshots to score, or the reverse for employer search); a **canonical skill vocabulary** is shared implicitly because both owning agents normalize to the same alias table; and **recommendation results** flow outward to the UI as ranked lists with explanations, never written back into profile stores. Conversely, the design withholds anything that would blur accountability: the Candidate Agent never sees unpublished employer drafts or recruiter notes; the Employer Agent sees only the match-relevant fields of a shortlisted candidate; and the Matchmaking Agent never receives credentials, authentication secrets, or raw feedback logs as ranking inputs.

3.4.3 Event communication

When a confirmed snapshot changes, the owning agent—and only that agent—embeds the canonical document text, upserts into its own vector store, and emits a profile- or job-updated event; the opposite store is untouched. The Matchmaking Agent never upserts; it queries both stores read-only, loading the query-side snapshot, retrieving candidate or job vectors, scoring each feasible pair with the configured strategy, attaching explanations, and sorting (the same pipeline runs in reverse for employer search). If a store lacks a fresh embedding, the owning agent must register a snapshot first—the matcher fabricates no missing state. Update events let the Matchmaking layer drop stale cached rankings for the affected entity and recompute on the next request.

3.4.4 Human control

Consequential outcomes stay with people. Confirming a resume or job edit triggers the owning agent's snapshot refresh and event; opening the matches page—or an employer's reverse search—invokes the Matchmaking Agent on the current snapshots, so nothing ranks on half-edited forms. Secondary actions (save, apply, shortlist, dismiss) are recorded as activity and do not alter embeddings unless the user later resaves. Because snapshots change only after confirmation, a person can review parsed fields and understand what will be compared before any recommendation is computed; administrators may inspect events for debugging but cannot override ownership through the UI.

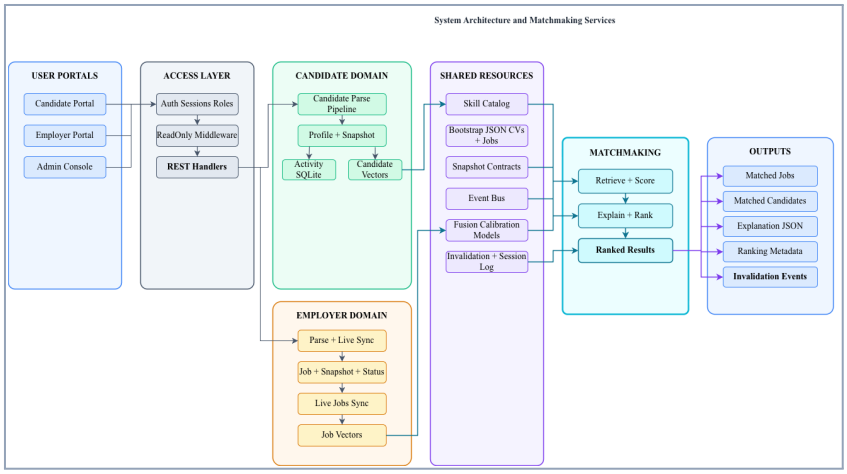

Fig. 7: Detailed view of the high-level architecture: the candidate lane (Job Seeker, Candidate Portal, Candidate Agent) and employer lane (Employer/Recruiter, Employer Portal, Employer Agent) communicate with the central Matchmaking Agent via snapshots and events and receive ranked matches back; an Admin/Evaluation Console sits below for fairness inspection, reset, and benchmarking.

3.5 End-to-End Workflow

Figure 4 traces the employer path; Figure 6 traces the candidate path. In both portal workflows, document ingestion is assistive: extracted fields populate review forms, users confirm or correct them, and only then does the owning agent register a snapshot and notify the matchmaking layer.

Detailed Workflow

Inputs: Candidate resumes or CVs; job descriptions; candidate preferences when available; employer constraints when available.

Outputs: Ranked candidate–job matches; matching scores; structured explanations and matched-skill summaries.

1. **Candidate Agent** receives a resume or CV from the candidate portal.
2. **Candidate Agent** preprocesses and parses the document into assistive form fields.
3. **Candidate Agent** extracts skills, experience, education, and stated preferences; the user confirms or edits the result.
4. **Candidate Agent** generates a candidate embedding from the confirmed canonical text.
5. **Candidate Agent** stores the candidate profile, versioned snapshot, and vector representation; emit a profile-updated event.
6. **Employer Agent** receives a job description from the employer portal.
7. **Employer Agent** preprocesses and parses the job description.
8. **Employer Agent** extracts role title, required skills, experience bounds, compensation band, location or remote policy, and hiring constraints; the user confirms or edits the result.
9. **Employer Agent** generates a job embedding from the confirmed canonical text.
10. **Employer Agent** stores the job profile, versioned snapshot, and vector representation; emit a job-updated event.
11. **Matchmaking Agent** retrieves candidate and job representations from both vector stores (read-only).
12. **Matchmaking Agent** computes semantic similarity and optional constraint-aware component scores (formulas in Section 5).
13. **Matchmaking Agent** ranks candidate–job pairs by the configured strategy (portal default: composite score).
14. **Matchmaking Agent** generates an explanation and matched-skill summary for each ranked pair.
15. **UI/Application Layer** displays ranked recommendations; the user explicitly chooses save, apply, shortlist, or contact.

The detailed workflow in Section 3.5 traces the end-to-end multi-agent matching process at the architectural level; score definitions appear in Section 5 and implementation paths in Section 4.

Candidates browse ranked jobs with explanation panels, then explicitly save or apply. Employers browse ranked candidates for a posting, then shortlist or contact through portal actions. Administrators inspect agent health and recent events in a separate console. Separate web experiences for candidates, employers, and administrators sit above the gateway; routing, endpoints, and default request parameters are specified in Section 4.

4 Implementation

This section summarizes how the architecture in Section 3 is realized in the prototype. Detailed component maps, data contracts, runtime defaults, and reproducibility artifacts are moved to the supplementary information to keep the main paper focused. Section 5 defines the scores and Section 6 reports outcomes; the detailed runtime pipeline diagram is provided in the supplementary information.

4.1 Runtime boundaries

The backend runs as a single FastAPI process wired through the repository bootstrap layer. Logical agents share a process and communicate through an in-process event bus rather than network RPC. Candidate and employer routes mutate only their owned records; the Matchmaking Agent consumes immutable snapshots and returns ranked lists with explanation payloads. Authentication uses role-scoped sessions for candidate, employer, and administrator portals, while benchmark and telemetry endpoints remain available to the admin console and tests.

At runtime, an owning agent parses an uploaded resume or job description, normalizes fields, embeds the resulting snapshot, and emits a profile-update event. The Matchmaking Agent then retrieves an exhaustive or ANN-filtered candidate set, rescales requested component scores, applies the configured ranking strategy, and attaches explanation fields for the portal. Lexical BM25/TF–IDF retrieval is retained for offline baselines rather than as the portal default. An optional live-job refresh path can replace the employer corpus from an external provider API, but all reported experiments use the frozen offline corpus.

The main user-facing flows are candidate onboarding/profile/matches/saved jobs, employer job management/reverse matching/applications, and administrator monitoring/manual inspection. Detailed route mappings are provided in the supplementary information.

4.2 Frontend implementation

The frontend is a React 19 single-page application built with Vite. It exposes role-specific candidate, employer, and administrator surfaces over the same session-cookie backend. Candidate views cover onboarding, profile review, match explanation, and saved jobs. Employer views cover job editing, application inspection, and reverse candidate search for owned postings. The administrator console exposes agent status, recent events, manual match controls, system configuration, and fairness summaries. The portal demonstrates end-to-end workflow feasibility; it is not evaluated as a live-user study.

4.3 Contracts, defaults, and reproducibility

The main implementation invariant is that editable portal records are converted into match-ready snapshots before scoring. Contracts are implemented as Pydantic models, and feedback records store user action context without rewriting embeddings. The portal default is deliberately conservative: fixed composite scoring, rules-based explanations, no calibration, no feedback boost, no cross-encoder rerank, and no automatic strategy switch. This default keeps online behavior interpretable and aligned with the score definitions in Section 5.

Offline evaluation does not depend on portal state. Scripts build snapshots from frozen files, run benchmark drivers, and compare selected metrics against committed expected values. The supplementary information lists the implementation map, contracts, runtime defaults, dependencies, and regeneration commands needed to reproduce the paper-facing runs.

5 Quality Metrics

This section defines the scores computed inside the Matchmaking Agent and the ranking, explanation, and fairness criteria used in offline evaluation. Implementation modules appear in Section 4; numeric outcomes appear in Section 6.

5.1 Evaluation corpus and protocol

Offline evaluation treats each resume as a query and each job posting as a candidate document. The labeled set comprises 30 resumes, 15 jobs, and 47 graded query–document pairs in `data/eval_pairs.json`. Relevance grades are $r \in \{0, 1, 2\}$ (none, partial, strong); queries with $r > 0$ form the relevant set for binary metrics, while nDCG uses the full graded scale.

The relevance labels were assigned manually by the project authors during demo-corpus construction; they are not inferred from portal clicks or hiring outcomes. Each candidate–job pair was judged against the structured snapshot fields used by the matcher: resume skills, experience level, job title, required skills, compensation constraints, and remote-policy fit. A strong label indicates that the candidate satisfies the core role requirements; a partial label indicates plausible fit with missing or weaker requirements; and a none label indicates no clear role fit. Final labels were chosen conservatively: when a pair was borderline, it was kept at partial or none rather than promoted to strong.

Unless a driver overrides `top_k`, metrics are macro-averaged at K=5 over all 30 queries. All benchmark drivers use the same metric routine so method comparisons differ only in the ranking function. Table 1 summarizes corpus statistics.

Table 1: Evaluation corpus statistics.

Statistic	Value
Task	Résumé → jobs (candidate-initiated retrieval)
Candidates (queries)	30
Job postings (corpus)	15
Graded query–document pairs	47
Queries with ≥ 1 relevant job	30
Relevance scale	0 (none), 1 (partial), 2 (strong)
Grade-2 (strong) labels	21
Grade-1 (partial) labels	26
Labels per query (min / max)	1 / 2
Embedding model	`all-MiniLM-L6-v2` (d=384)
Label source	`data/eval_pairs.json` v1.0

Starter labeled set for offline benchmarks; not derived from live portal traffic.

5.2 Matching scores

A resume and a job snapshot pass through embedding similarity, skill overlap, and the experience, compensation, and remote signals, which combine into a weighted

composite score and a structured explanation that drives the ranked recommendation. All component scores are normalized to $[0, 1]$. Let $\mathbf{e}_c, \mathbf{e}_j \in \mathbb{R}^d$ denote candidate and job snapshot embeddings (d=384), and let A and B be canonical skill sets on the resume and required job skills, respectively.

The semantic score compares resume and job-description embeddings. Cosine mode is the portal default:

$$s_{\text{sem}}^{\cos} = \frac{\mathbf{e}_c \cdot \mathbf{e}_j}{\|\mathbf{e}_c\| \, \|\mathbf{e}_j\|}. \tag{1}$$

Euclidean mode maps distance to similarity:

$$s_{\text{sem}}^{\text{euc}} = \frac{1}{1 + \|\mathbf{e}_c - \mathbf{e}_j\|_2}. \tag{2}$$

The skill score compares resume skills against job requirements. Jaccard mode is the portal default:

$$s_{\text{skl}}^{\text{jac}} = \begin{cases} |A \cap B| / |A \cup B| & \text{if } A \neq \emptyset \text{ and } B \neq \emptyset, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

Embedding mode averages, for each required skill $b \in B$, the maximum cosine between b's skill phrase embedding and any resume skill embedding:

$$s_{\text{skl}}^{\text{emb}} = \frac{1}{|B|} \sum_{b \in B} \max_{a \in A} \cos\!\left(\mathbf{e}_a^{(\text{skill})}, \mathbf{e}_b^{(\text{skill})}\right). \tag{4}$$

A two-signal blend combines semantic and skill scores with weight α (default α=0.7):

$$s_{\text{mm}} = \alpha \, s_{\text{sem}} + (1 - \alpha) \, s_{\text{skl}}. \tag{5}$$

The composite score is a fixed weighted sum over six interpretable components—semantic, skills, title token overlap, experience fit, compensation fit, and remote preference—then clipped to $[0, 1]$:

$$\begin{gathered} s_{\text{final}} = \text{clip}_{[0,1]}\left(\sum_k w_k s_k \right), \\ (w_{\text{sem}}, w_{\text{skl}}, w_{\text{tit}}, w_{\text{exp}}, w_{\text{comp}}, w_{\text{rem}}) = (0.28, 0.27, 0.10, 0.15, 0.10, 0.10). \end{gathered} \tag{6}$$

Reciprocal rank fusion (RRF), used in comparison experiments, merges m ranked lists without retraining:

$$\text{RRF}(x) = \sum_{i=1}^{m} \sum_{r} \mathbb{1}[\mathcal{L}_i[r] = x] \cdot \frac{1}{k + r}, \tag{7}$$

with smoothing constant k=60 by default. Optional constraints, Platt calibration, learned fusion, feedback boosts, cross-encoder reranking, and ANN shortlist sweeps are evaluated as experimental branches; portal defaults keep them disabled.

5.3 Ranking metrics

Given ranked list π_q for query q, graded relevance map $\mathrm{rel}_q(\cdot)$, and relevant set $\mathcal{R}_q = \{j : \mathrm{rel}_q(j) > 0\}$, let $\pi_q[:K]$ denote the top-K prefix.

Precision@K is the fraction of the top-K slots occupied by relevant jobs:

$$\mathrm{P@}K(q) = \frac{1}{K} \sum_{j \in \pi_q[:K]} \mathbb{1}[j \in \mathcal{R}_q]. \tag{8}$$

Recall@K is the fraction of all relevant jobs for the query retrieved within the top-K:

$$\mathrm{R@}K(q) = \frac{1}{|\mathcal{R}_q|} \sum_{j \in \pi_q[:K]} \mathbb{1}[j \in \mathcal{R}_q]. \tag{9}$$

Normalized discounted cumulative gain rewards higher grades at higher ranks. For graded relevance:

$$\mathrm{DCG@}K(q) = \sum_{i=1}^{K} \frac{2^{\mathrm{rel}_q(j_i)} - 1}{\log_2(i+1)}, \tag{10}$$

$$\mathrm{nDCG@}K(q) = \frac{\mathrm{DCG@}K(q)}{\mathrm{DCG@}K(q^*)}, \tag{11}$$

where π_{q^*} is the ideal ranking that sorts jobs by rel_q descending.

MRR treats relevance as binary ($r > 0$) and measures how early the first relevant job appears:

$$\mathrm{RR}(q) = \begin{cases} 1/\mathrm{rank}_q(j^*) & \text{if a relevant job } j^* \text{ appears at rank } \mathrm{rank}_q(j^*), \\ 0 & \text{otherwise.} \end{cases} \tag{12}$$

Macro-averaged MRR $= \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathrm{RR}(q)$.

MAP averages precision at each relevant hit, normalized by $|\mathcal{R}_q|$:

$$\mathrm{AP}(q) = \frac{1}{|\mathcal{R}_q|} \sum_{r=1}^{|\pi_q|} \mathrm{P@}r(q) \cdot \mathbb{1}[\pi_q[r] \in \mathcal{R}_q], \tag{13}$$

where $\mathrm{P@}r(q)$ is precision computed over the length-r prefix. Drivers report macro-averaged P@K, R@K, MRR, nDCG@K, and MAP unless noted otherwise.

5.4 Benchmark comparison criteria

The benchmark suite evaluates, on the *same* labeled corpus: lexical baselines, dense strategies, composite and component ablations, fusion variants, optional cross-encoder reranking, and ANN shortlist sweeps. Primary comparison metrics are macro-averaged P@5, R@5, MRR, nDCG@5, and MAP at K=5.

Paired bootstrap tests resample queries with replacement (N=5000, seed 42) and compare method pairs on nDCG@K and MRR; significant comparisons are reported in Section 6.3.

Continuous integration recomputes selected rankings and requires:

$$\left|\widehat{\text{nDCG@5}}_{\text{run}} - \text{nDCG@5}_{\text{expected}}\right| \leq 0.04, \tag{14}$$

against committed values in `data/expected/paper_progression_summary.json` (`tests/benchmarks/test_eval_regression.py`).

From the top-P=10 multimodal pool per query, hard negatives are high-scoring jobs outside the labeled relevant set:

$$\mathcal{H}_q = \{j \in \pi_q[:P] : j \notin \mathcal{R}_q\}_{|\cdot|\leq 5}\,. \tag{15}$$

Success criterion: zero overlap between $\mathcal{H}_q$ and declared relevant jobs across all queries (label-set sanity, not ranking quality).

ANN shortlist latency is summarized in Section 6.5.

5.5 Explanation quality criteria

Explainability is evaluated offline on ranked pairs by comparing rule-based bullets (portal default) with a grounded template explainer. Faithfulness requires explanations to cite available matched or missing skills, avoid hallucinated skills, and align qualitative claims with the underlying score breakdown. Specificity measures whether bullets contain concrete skill references rather than generic text. For synthetic profile variants, consistency is measured with bullet-set Jaccard similarity and matched-skill equality.

Aggregate explanation-quality results are summarized in Section 6.6.

5.6 Fairness and sanity metrics

Fairness checks are proxy-based sanity audits on the synthetic demo corpus; they do not infer protected attributes from real users. Queries are grouped by experience tier and remote preference to compute selection-rate disparate-impact ratios. A separate counterfactual audit applies controlled edits to ten fabricated profile pairs and records rank shifts, score deltas, and explanation drift. Numeric outcomes appear in Section 6.6.

6 Results and Discussion

This section reports benchmark results on the demo corpus defined in Section 5, compares ranking methods, and discusses the limits of the evidence. All figures come from committed benchmark artifacts; they do not predict hiring outcomes at production scale.

6.1 Evaluation scope and protocol

We evaluated *resume-to-job matching*: for each of 30 sample resumes, the system ranks 15 job postings and is judged at cutoff K=5. The 47 manually labeled pairs

use the relevance scale defined in Section 5.1. Metrics are macro-averaged across all 30 resumes so that no single profile dominates the score.

6.2 Baseline methods

The first comparison contrasts lexical baselines, dense semantic matching, skill overlap, hybrid semantic–skill matching, and the portal-default composite scorer on the same labeled set. Multimodal weighted scoring achieves the highest ranking quality (nDCG@5 = 0.924) among the methods listed, while BM25 and TF–IDF remain close (nDCG@5 $\approx$ 0.89–0.90). On this small job pool, lexical baselines remain competitive, so gains from dense and hybrid matching are measurable but modest. Semantic similarity alone (nDCG@5 = 0.878) underperforms both keyword baselines and multimodal blends, supporting the use of explicit skills alongside embeddings. The full method-comparison table is provided in the supplementary information.

6.3 Main results

The progression study (Table 2) ranks every job for every resume and tracks how design choices accumulate. The best single configuration—multimodal scoring with soft skill embeddings—reaches nDCG@5 = 0.969 and recall@5 = 1.00, meaning every resume retrieves all of its labeled relevant jobs within the top five. Plain semantic cosine reaches nDCG@5 = 0.911; adding richer text templates nudges it to 0.922 but still trails the best hybrid. Merging four lists with RRF (0.935) helps diversity but does not beat the strongest single hybrid on this corpus. Cross-encoder reranking improves over bare semantic search (0.939) yet still falls short of soft-embed multimodal (0.969) while adding far more compute.

Table 2: Paper progression benchmark (regression baselines, K=5).

Method	P@5	R@5	nDCG@5
Multimodal soft embed w=0.7	0.313	1.000	0.969
Soft embed + cross-encoder (pool=10)	0.307	0.983	0.939
RRF ensemble (4 lists)	0.293	0.950	0.935
Multimodal Jaccard w=0.7	0.293	0.950	0.933
Semantic cosine (rich templates)	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901

Source: `data/expected/paper_progression_summary.json`. Produced by `benchmarks/paper_progression.py`; gated by `tests/benchmarks/test_eval_regression.py` ($\pm$0.04 nDCG tolerance). Values can differ from the method-comparison run when template or cache flags differ.

Learned logistic-regression fusion ties the best progression run (nDCG@5 = 0.968), while fixed multimodal blending and hierarchical skill weighting both score 0.924. Applying hard constraints after scoring lowers nDCG@5 to 0.908 because filters can remove jobs that annotators marked as partially relevant. Full fusion and model-parameter tables are provided in the supplementary information.

Statistical significance. Paired bootstrap tests (N=5000, seed 42) confirm the hybrid gains are not noise: the full composite significantly improves nDCG@5 over semantic-only ranking (Δ=+0.071, p=0.048), and multimodal blending and RRF both significantly beat semantic cosine (p=0.010 and p=0.009). Lexical baselines (BM25, TF–IDF) do not differ significantly from semantic cosine (p>0.25), consistent with their close scores on this small pool.

The main finding is modest but consistent: matching improves when the system combines language semantics with explicit skills, and the highest scores arise from hybrid signals rather than either channel alone. No method achieves high precision—at best, roughly three in ten top-five slots are relevant (P@5 $\approx$ 0.31)—because the corpus is small, labels are sparse, and several jobs can appear plausible for the same resume. We therefore characterize JobMatch as *assistive* ranking with explainable reasons, not as an automated hiring decision system.

6.4 Multi-agent state sharing and usability

When a candidate edits a resume or an employer revises a job description, the owning agent rebuilds a versioned snapshot and shared embedding, then notifies the Matchmaking layer through the event bus (Section 3–Section 4). The event-driven path addresses a usability problem seen in monolithic ATS tools: rankings that stay stale after a profile change, or that change without a visible cause. Role-separated agents also keep each side's data under its owner's control, which supports trust and auditability even when two users see the same match score from the neutral Matchmaking agent. The benchmarks validate that the shared scoring core ranks sensibly under manual labels; they do not yet prove that multi-agent coordination reduces time-to-hire or increases applicant satisfaction in the field.

6.5 Strengths

Any method that combines text meaning with skills beats semantic-only search (nDCG@5 0.878 vs. 0.917–0.949 for richer composites). The portal-default composite reaches nDCG@5 = 0.949; semantic+skills alone reaches 0.917, while using experience, compensation, or location alone yields poor rankings (nDCG@5 $\leq$ 0.39). Detailed ablation rows are provided in the supplementary information.

Lightweight scorers complete in well under one millisecond per resume on 15 jobs; the full composite bi-encoder run averages $\sim$0.4 ms per query. That headroom matters for interactive portals where a user expects refreshed matches after saving a profile. Cross-encoder reranking is much slower ($\sim$141.7 ms/query) and lowers nDCG@5 in the committed report, so it remains disabled by default. The full latency table is provided in the supplementary information.

Hard-negative mining finds zero overlap between declared relevant jobs and 150 high-scoring mined negatives, supporting internal label consistency. The live portal defaults to composite scoring because each channel maps to a human-readable explanation, even though some research variants score slightly higher offline.

6.6 Limitations

The evaluation has four main limits: the labeled corpus is small, the fairness audit is synthetic, no live-user study was run, and JobMatch remains a research prototype rather than a validated deployment. Thirty resumes and fifteen jobs make recall@5 easy to saturate—many methods reach $\geq$0.93—so differences in nDCG are informative but narrow. Results may not transfer to employers with thousands of open roles or to domains unlike our demo profiles. Even the best methods place relevant jobs in only about three of every ten top-five slots (P@5 $\approx$ 0.31). Users will still see imperfect suggestions and must rely on explanations and their own judgment.

Cross-encoder reranking is $\sim$340$\times$ slower than composite bi-encoder scoring on this setup and *lowers* nDCG@5 to 0.834 (a 0.108 drop from the bi-encoder baseline) in the committed report—consistent with leaving it disabled by default. Learned fusion and calibration fit on the same labeled pairs they are judged against, which risks overfitting.

The fairness audit is narrow: ten fabricated profile pairs undergo controlled name, summary, or metadata edits. Seven pairs trigger flags, all with the top-1 job stable, mostly due to small rank shifts or score changes; the maximum score shift is 0.017. A complementary proxy-group check (`fairness_eval.py`) reports selection-rate disparate-impact ratios of 0.82 across experience tiers and 0.75 across remote preference (1.0 is parity), committed to `benchmark_outputs/fairness_eval.json`. These checks are useful engineering probes, but they do not certify demographic fairness or substitute for audits on larger labeled populations.

We did not measure interview rates, time-to-fill, applicant satisfaction, recruiter workload, or live-user behavior in the portals. The implementation should therefore be read as a research prototype with reproducible offline evidence, not as a validated deployment study or production hiring system. Both explainers are perfectly consistent across synthetic profile variants and never make unsupported component claims, but they cite an explicit matched or missing skill in only about a quarter of bullets, so most explanations are flagged for low specificity—motivating richer explanation templates. Detailed fairness and explainability audit tables are provided in the supplementary information.

6.7 Implications for users

JobMatch is designed as an assistive representative: maintain a structured profile, inspect why a job ranked highly or poorly, and decide whether to save or apply. The evaluation indicates that suggestions are most reliable when the system weights both resume text and listed skills—consistent with how applicants skim postings for fit. Top-five lists are neither exhaustive nor authoritative; with P@5 near 0.31, several shown jobs may be weak fits, and relevant openings can be missed when the job pool exceeds fifteen roles.

Employers. The employer portal provides symmetric reverse matching: ranked candidates with component-level reasons instead of an opaque score. Hybrid ranking supports recruiters who weight keywords and hiring managers who read career narrative. Hard post-filters can remove partially suitable candidates from the graded top

five, so the portal keeps constraints optional and off by default. When a job description changes, the Employer Agent update path refreshes rankings without re-upload to a separate scoring service.

Both sides benefit when software explains its suggestions and leaves hire, reject, and apply decisions with users. The benchmarks support that design on a demo corpus: combined signals rank better than single-signal search, explanations align with composite channels, and the multi-agent layout keeps profile edits on the owning side while a neutral Matchmaking Agent serves both portals. They do not establish production gains on hiring outcomes; that requires larger labeled datasets, portal-default ablations, and field studies.

7 Conclusion and Future Scope

7.1 Conclusion

Job seekers and recruiters still interact through software that ranks quickly but often explains little. JobMatch addresses this trust problem with a role-separated multi-agent architecture: the Candidate Agent owns resumes and candidate snapshots, the Employer Agent owns job descriptions and job snapshots, and the read-only Matchmaking Agent scores pairs and returns ranked lists with structured explanations. Candidate, employer, and administrator portals sit on top of this layout so each party manages its own data while sharing only what matching requires.

We evaluated the Matchmaking agent offline on a labeled demo corpus—30 resumes, 15 jobs, and 47 graded relevance pairs—using the metrics defined in Section 5 and reported in Section 6. Hybrid rankers that combine semantic text similarity with explicit skill overlap beat semantic search alone, while keyword methods such as BM25 remain competitive on this small pool. The best research configurations reach nDCG@5 ≈ 0.97 and recall@5 $= 1.00$, but precision@5 stays near 0.31. The live portal therefore defaults to a fixed composite score whose channels map directly to user-visible explanations, rather than to the single highest offline nDCG variant. These results are reproducible from committed benchmark artifacts; they characterize ranking behavior on a controlled corpus, not hiring quality in production.

JobMatch is intentionally built as **decision-support software**, not an automated hiring replacement. It suggests, ranks, and explains; users save, apply, shortlist, or pass. The contribution is an integrated research prototype: role-aware agents, shared snapshots, explainable composite ranking, role-separated portals, optional external job ingestion, and offline evaluation tied to the same scoring code the gateways invoke. Whether this design improves real-world trust, time-to-shortlist, or fairness is not settled by laboratory metrics alone and remains for field study.

7.2 Future scope

Future work should first expand evaluation beyond the small demo corpus. Useful next steps include expert-labeled resume–job pairs, larger held-out corpora, deployment-scale latency sweeps across Chroma and Qdrant, and field studies with recruiters

and job seekers to measure comprehension, trust, and decision behavior after viewing match explanations.

The implementation also leaves several engineering directions open. Large language models could improve extraction from noisy resumes and informal job descriptions, provided users confirm fields before snapshots enter the vector store. Cross-encoder reranking should be retested on larger corpora and tighter shortlists, since it reduced nDCG@5 and added substantial latency on the current demo corpus. The optional live-job sync path needs authenticated admin controls, rate limiting, and provider credential management before untrusted deployment.

Finally, agent ownership raises fairness, privacy, and feedback-loop questions. The current counterfactual and proxy-group audits are engineering checks, not demographic fairness certification. Future work should add larger fairness audits, field-level minimization for profile sharing, and carefully bounded feedback policies that improve relevance without obscuring the reasons shown to users.

The design goal is unchanged: agents keep profiles current, the Matchmaking layer shows its work, and people retain final hiring decisions.

Acknowledgments. The authors thank Thapar Institute of Engineering and Technology for institutional support and Dr Parteek Kumar, Associate Professor, Washington State University, Pullman, WA, for research supervision, technical direction, and manuscript guidance. Collaborators who reviewed portal workflows and benchmark reproducibility checks are also acknowledged.

Declarations

Funding

This work was supported by the NVIDIA Academic Grant Program through an unrestricted gift of 32,000 NVIDIA A100 GPU-hours on the Brev cloud platform.

Competing interests

The authors declare no competing interests.

Ethics approval

Not applicable. Evaluation uses synthetic and manually curated demo profiles; no human subjects recruitment data were collected for this study.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The project repository includes the demo corpus under `data/` and benchmark outputs under `backend/benchmark_outputs/` and `docs/research/evaluation/`. The corpus files are `cvs.json`, `jobs.json`, and `eval_pairs.json`; metrics can be regenerated with the repository benchmark scripts.

Code availability

Source code for the platform, benchmark drivers, and regression tests is available in the JobMatch repository at the submission tag. Key entry points include the backend gateway and agents, the matching core, the React frontend, and benchmark regression tests. The repository is hosted at https://github.com/Harsh23Kashyap/Job-Matching-Agentic.

Author contributions

Harsh Kashyap and **Taranumpreet Kaur Wasu** contributed equally to conceptualization, software implementation, evaluation, and manuscript preparation. **Dr Parteek Kumar**, Associate Professor, Washington State University, Pullman, WA, served as research supervisor and provided guidance on technical direction and manuscript preparation.

Prior publication

This manuscript has not been published previously and is not under consideration elsewhere. No portion of the text has appeared in a preprint with colored layout styling.

References

[1] Rosenfeld, A., Richardson, A.: Explainability in human–agent systems. Autonomous Agents and Multi-Agent Systems **33**(6), 673–705 (2019) https://doi.org/10.1007/s10458-019-09408-y

[2] Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, Cambridge, UK (2008)

[3] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. In: Foundations and Trends in Information Retrieval, vol. 3, pp. 333–389 (2009)

[4] Malinowski, J., Keim, T., Wendt, O., Weitzel, T.: Matching people and jobs: A bilateral recommendation approach. In: Proceedings of the 39th Annual Hawaii International Conference on System Sciences (HICSS-39). IEEE, ??? (2006). https://doi.org/10.1109/HICSS.2006.266

[5] Kleinerman, A., Rosenfeld, A., Kraus, S.: Providing explanations for recommendations in reciprocal environments. In: Proceedings of the 12th ACM Conference on Recommender Systems (RecSys 2018), pp. 137–145. ACM, ??? (2018). https://doi.org/10.1145/3240323.3240362

[6] Xia, B., Yin, J., Xu, J., Li, Y.: WE-Rec: A fairness-aware reciprocal recommendation based on walrasian equilibrium. Knowledge-Based Systems **182**, 104857 (2019) https://doi.org/10.1016/j.knosys.2019.07.028

[7] Wooldridge, M.: An Introduction to MultiAgent Systems, 2nd edn. Wiley, Chichester, UK (2009)

[8] Dell'Anna, D., Murukannaiah, P.K., Yurrita, M., Dudzik, B., Grossi, D., Jonker, C.M., Oertel, C., Yolum, P.: From human teams to hybrid intelligence teams: identifying, characterizing, and evaluating foundational quality attributes. Autonomous Agents and Multi-Agent Systems (2026) https://doi.org/10.1007/s10458-025-09730-8

[9] Omicini, A., Ricci, A., Viroli, M.: Artifacts in the A&A meta-model for multi-agent systems. Autonomous Agents and Multi-Agent Systems **17**(3), 432–456 (2008) https://doi.org/10.1007/s10458-008-9053-x

[10] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)

[11] Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Proceedings of SIGIR, 758–759 (2009)

[12] Platt, J.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 61–74 (1999)

[13] Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. Proceedings of KDD, 1135–1144 (2016)

[14] Yan, E., Burattini, S., Hübner, J.F., Ricci, A.: A multi-level explainability framework for engineering and understanding BDI agents. Autonomous Agents and Multi-Agent Systems **39**(1), 9 (2025) https://doi.org/10.1007/s10458-025-09689-6